

TLN – Parte 3: Di Caro

Similarità, Topic Modeling e RAG

A.A. 2025/26 – UniTO Magistrale Informatica

Riassunto per lo studio

Indice

1 Introduzione: Contenuto vs. Forma Lessicale

Due direzioni nel lessico:

- **Semasiologica** (Forma $\rightarrow$ Contenuto): dato un lemma, cerco il suo significato. Direzione classica del dizionario.
- **Onomasiologica** (Contenuto $\rightarrow$ Forma): data una descrizione/definizione, trovo la parola corretta. Problema di *content-to-form retrieval*.

Il percorso del corso copre tre macro-temi:

1. **Similarità lessicale/semantica** (Lab 1–2): misure Jaccard e TF-IDF coseno su definizioni umane
2. **Topic Modeling neurale** (Lab 4): pipeline BERTopic per scoperta automatica di argomenti
3. **RAG ibrido** (Lab 5): retrieval aumentato con FAISS semantico + BM25 lessicale + RRF + LLM

2 I Quattro Tipi Concettuali

Nei lab 1 e 2 si lavora su quattro concetti scelti lungo due assi: **concretezza** (concreto/astratto) e **specificità** (generico/specifico).

Tipo	Sigla	Esempio	Concretezza	Specificità
Concrete Generic	CG	Albero (Tree)	Concreto	Generico
Concrete Specific	CS	Teiera (Teapot)	Concreto	Specifico
Abstract Generic	AG	Musica (Music)	Astratto	Generico
Abstract Specific	AS	Etica (Ethics)	Astratto	Specifico

Caratteristiche per tipo:

- **CG (Albero)**: referente fisico, classe ampia. Definizioni usano vocabolario condiviso (radici, tronco, rami, foglie). Alta coerenza cross-individuale.
- **CS (Teiera)**: referente fisico specifico, funzione primaria. Vocabolario concentrato su funzione (infusione, versare).
- **AG (Musica)**: nessun referente fisico, classe ampia. Definizioni variano enormemente: operative, sensoriali, emotive.
- **AS (Etica)**: nessun referente fisico, dominio filosofico. Alta influenza culturale e contestuale. Eterogeneità massima.

3 Similarità Lessicale e Semantica (Lab 1)

3.1 SimLex – Indice di Jaccard

$$\text{SimLex}(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

dove A e B sono insiemi di parole lemmatizzate e filtrate (stopwords rimosse) delle due definizioni.

Cosa misura: sovrapposizione *lessicale* pura – due testi con stesso significato ma parole diverse → score basso.

Pipeline di preprocessing:

1. Lowercase
2. Rimozione punteggiatura
3. Tokenizzazione (NLTK `word_tokenize`)
4. Rimozione stopwords (NLTK `stopwords.words('italian')`)
5. Lemmatizzazione
6. Conversione in `set`

```

1 from nltk.corpus import stopwords
2 from nltk.tokenize import word_tokenize
3 import re, string
4
5 def preprocess_to_set(text, lang='italian'):
6     text = text.lower()
7     text = re.sub(r'[^\w\s]', '', text)
8     tokens = word_tokenize(text, language=lang)
9     stop = set(stopwords.words(lang))
10    tokens = [t for t in tokens if t not in stop and len(t) > 1]
11    return set(tokens) # set per Jaccard
12
13 def simlex(defA, defB):
14     A = preprocess_to_set(defA)
15     B = preprocess_to_set(defB)
16     if not A and not B:
17         return 0.0
18     return len(A & B) / len(A | B)

```

3.2 SimSem – Coseno TF-IDF

$$\text{SimSem}(A, B) = \cos(\vec{v}_A, \vec{v}_B) = \frac{\vec{v}_A \cdot \vec{v}_B}{\|\vec{v}_A\| \cdot \|\vec{v}_B\|}$$

dove $\vec{v}$ è la rappresentazione TF-IDF del testo.

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) \times \log \frac{N}{\text{df}(t)}$$

Cosa misura: prossimità *semantica* – cattura sinonimi e termini correlati che il semplice overlap non coglie.

```

1 from sklearn.feature_extraction.text import TfidfVectorizer
2 from sklearn.metrics.pairwise import cosine_similarity
3
4 def simsem(defA, defB):
5     corpus = [preprocess_text(defA), preprocess_text(defB)]
6     vectorizer = TfidfVectorizer()
7     tfidf_matrix = vectorizer.fit_transform(corpus)
8     return cosine_similarity(tfidf_matrix[0:1], tfidf_matrix[1:2])
9     [0][0]

```

3.3 Confronto tra coppie – tutte le combinazioni

Il codice usa `itertools.combinations` per calcolare similarità tra tutte le coppie di concetti:

```

1 import itertools, csv
2
3 concetti = ['Musica', 'Etica', 'Albero', 'Teiera']
4 definizioni = {c: [...] for c in concetti} # lista def per c
5
6 risultati = []
7 for c1, c2 in itertools.combinations(concetti, 2):
8     for def1, def2 in zip(definizioni[c1], definizioni[c2]):
9         sl = simlex(def1, def2)
10        ss = simsem(def1, def2)
11        risultati.append({'concetto1': c1, 'concetto2': c2,
12                          'simlex': sl, 'simsem': ss})
13
14 # salva su CSV
15 with open('risultati_similarita.csv', 'w') as f:
16     writer = csv.DictWriter(f, fieldnames=risultati[0].keys())
17     writer.writeheader()
18     writer.writerows(risultati)

```

3.4 Risultati Lab 1

Concetto	SimLex (Jaccard)	SimSem (Coseno)
Albero (CG)	0.084	0.077
Teiera (CS)	0.057	0.062
Etica (AS)	0.034	0.041
Musica (AG)	0.034	0.036

Osservazioni chiave:

- Valori molto bassi ovunque (3–8%): enorme variabilità del linguaggio naturale anche per lo stesso referente.
- **Concreto > Astratto**: i concetti fisici hanno referenti che vincolano il vocabolario.
- **Per i concetti astratti, SimSem > SimLex**: il modello vettoriale allinea sinonimi e termini correlati che la Jaccard pura non vede. Questo dimostra il valore aggiunto della rappresentazione semantica.

- Etica: sebbene `simlex = simsem = 0.034`, le definizioni condividono *struttura concettuale* nonostante parole diverse.

4 Content-to-Form: Ricerca Onomasiologica (Lab 2)

4.1 Il Task

Ricerca onomasiologica: data una definizione in linguaggio naturale, recuperare il *synset* WordNet corretto.

Input: *"Branch of philosophy that studies human behavior by trying to define what is right and wrong"*

Output: `ethics.n.01`

È l'inverso della classica consultazione dizionario: si parte dal concetto/significato per trovare la forma lessicale.

4.2 Procedura Lab 2

1. **Selezione candidati:** per ogni concetto target, 5 synset WordNet candidati con relative glosse (incluse forme alternate e iperonimi)
2. **Preprocessing:** lowercase, rimozione punteggiatura, filtro stopwords su definizione utente E glosse
3. **Vettorizzazione TF-IDF:** l'intero corpus = [definizione_pulita] + [glosse_candidate_pulite]
4. **Coseno:** similarità tra definizione e ogni glossa candidata
5. **Predizione:** synset con score massimo = risposta del sistema
6. **Valutazione:** confronto con synset target

```

1 from sklearn.feature_extraction.text import TfidfVectorizer
2 from sklearn.metrics.pairwise import cosine_similarity
3
4 # Per ogni concetto: definizione umana + 5 glosse candidate
5 concetti = {
6     'Albero': {
7         'target': 'tree.n.01',
8         'definizioni': [...], # definizioni degli studenti
9         'candidati': {
10             'tree.n.01': 'a tall perennial woody plant having...',
11             'plant.n.01': 'a living organism lacking the power...',
12             # ... altri 3 candidati
13         }
14     },
15     # ... altri concetti
16 }
17
18 def content_to_form(defn, glosse_dict):
19     glosse = list(glosse_dict.values())
20     synsets = list(glosse_dict.keys())
21     corpus = [clean(defn)] + [clean(g) for g in glosse]
22

```

```

23 vec = TfidfVectorizer()
24 mat = vec.fit_transform(corpus)
25 scores = cosine_similarity(mat[0:1], mat[1:])[0]
26 best_idx = scores.argmax()
27 return synsets[best_idx], scores[best_idx]

```

4.3 Struttura Genus-Differentia

Schema Aristotelico: la migliore strategia definitoria è *genus proximum + differentia specifica*:

- **Genus:** categoria sovraordinata (spesso corrisponde all'ipernimo WordNet)
- **Differentia:** proprietà che distinguono il concetto dagli altri della stessa classe

Esempio efficace: *"Branch of philosophy [genus] that studies moral values [differentia]"*

Corrisponde quasi letteralmente alla glossa `ethics.n.01`: *"the philosophical study of moral values and rules"*

4.4 Risultati Lab 2

Concetto	Tipo	Accuracy	Score Avg	Score Max
Albero (CG)	Concreto/Generico	89.3%	0.1025	0.2919
Teiera (CS)	Concreto/Specifico	71.4%	0.1272	0.3063
Etica (AS)	Astratto/Specifico	42.9%	0.1524	0.6295
Musica (AG)	Astratto/Generico	32.1%	0.1298	0.3981

Aggregato per dimensione:

Dimensione	Accuracy	Score Avg
Concreto	80.4%	0.1149
Astratto	37.5%	0.1411
Generico	60.7%	0.1162
Specifico	57.1%	0.1398

Analisi:

- Concetti concreti: $2.1\times$ più recuperabili degli astratti. Il referente fisico vincola il vocabolario.
- Definizione migliore per Etica (score 0.63): "Branch of philosophy that studies human behavior by trying to define what is right and what is wrong, often influenced by historical and sociocultural context." – ha genus-differentia esplicita.
- TF-IDF fallisce su sinonimia: richiede embedding contestuali per i concetti astratti.

5 Topic Modeling con BERTopic (Lab 4)

5.1 Architettura della Pipeline

Pipeline BERTopic (4 stadi):

Documenti $\xrightarrow{1. \text{ Embedding}} \xrightarrow{2. \text{ UMAP}} \xrightarrow{3. \text{ HDBSCAN}} \xrightarrow{4. \text{ c-TF-IDF}}$ Topic interpretabili

5.2 Stadio 1: Embedding

Ogni documento viene rappresentato come vettore denso semantico tramite un *Sentence-Transformer*.

Modello	Dimensioni	Caratteristiche
thenlper/gte-small	384	Qualità alta, lento su CPU (13.484s)
all-MiniLM-L6-v2	384	Compatto, veloce su CPU, comparabile

5.3 Stadio 2: UMAP (Riduzione Dimensionale)

UMAP (Uniform Manifold Approximation and Projection): riduce le dimensioni preservando sia struttura locale che globale.

- **n_components**: dimensioni output (meno = più veloce, meno dettaglio semantico)
- **n_neighbors**: dimensione del vicinato locale (basso = focus locale; alto = focus globale)
- **min_dist**: distanza minima tra punti (0.0 per clustering; più alto per visualizzazione)
- **metric**: “cosine” per embedding normalizzati

Vantaggio su t-SNE: preserva struttura globale oltre a quella locale.

5.4 Stadio 3: HDBSCAN (Clustering)

HDBSCAN (Hierarchical Density-Based Spatial Clustering of Applications with Noise):

- Identifica cluster di forma arbitraria (non solo sferici)
- Assegna automaticamente outlier all’etichetta -1
- **Non richiede** di specificare il numero di cluster a priori
- Parametro chiave: **min_cluster_size** – dimensione minima cluster

5.5 Stadio 4: c-TF-IDF (Estrazione Keywords Topic)

$$\text{c-TF-IDF}(t, c) = \frac{\text{freq}(t, c)}{|c|} \times \log \frac{N_{\text{docs}}}{|\{d : t \in d\}|}$$

dove c è il cluster (topic), non il singolo documento.

Differenza da TF-IDF classico: la frequenza viene calcolata *all'interno del cluster* come contesto, non del singolo documento. Produce keyword interpretabili e specifiche per ogni topic.

5.6 Configurazioni Testate

	Modello	UMAP	HDBSCAN	Effetto
Config 1 (base)	gte-small	comp=5, neigh=15	mcs=50	Bilanciato
Config 2 (fine)	MiniLM-L6	comp=10, neigh=10	mcs=30	Granulare
Config 3 (coarse)	MiniLM-L6	comp=5, neigh=30	mcs=150	Grossolano (fallimento)

5.7 Risultati Comparativi

Metrica	Config 1	Config 2	Config 3
# Topic (escl. -1)	151	171	49
Outlier count	14.357	12.747	14.899
Outlier %	31.9%	28.4%	33.1%
Runtime	~3h 47m	~17m 30s	~11m
Topic 0 keywords	speech, asr, recognition	dialogue, response, conv.	dialogue, the, to, and

Analisi failure mode Config 3:

- `min_cluster_size=150`: cluster troppo grandi e tematicamente eterogenei
- Il c-TF-IDF non riesce a estrarre segnale discriminativo: le stopwords emergono nel top-5
- Solo 49 topic (vs. 151-171 degli altri) con outlier percentuale paradossalmente più alta
- **Lezione**: parametri troppo grossolani → perdita di interpretabilità

Config 2 (raccomandata per CPU):

```

1 from sentence_transformers import SentenceTransformer
2 from bertopic import BERTopic
3 from umap import UMAP
4 from hdbscan import HDBSCAN
5
6 embedding_model = SentenceTransformer("all-MiniLM-L6-v2")
7 umap_model = UMAP(n_components=10, n_neighbors=10,
8                   min_dist=0.0, metric='cosine')
9 hdbscan_model = HDBSCAN(min_cluster_size=30,
10                          cluster_selection_method='eom',
11                          prediction_data=True)
12
13 topic_model = BERTopic(
14     embedding_model=embedding_model,
15     umap_model=umap_model,
16     hdbscan_model=hdbscan_model,
17     language="english",
18     verbose=True

```



```

19 )
20
21 topics, probs = topic_model.fit_transform(docs)
22 print(topic_model.get_topic_info().head(10))

```

Dataset: MaartenGr/arxiv_nlp (HuggingFace) – 44.949 abstract di paper NLP.

6 RAG: Retrieval-Augmented Generation

6.1 Architettura Generale

RAG combina un *retriever* (recupera documenti rilevanti) con un *generatore* (LLM che risponde basandosi sul contesto):



Vantaggi rispetto a LLM standalone:

- Accesso a conoscenza esterna e aggiornabile senza retraining
- Riduzione delle *allucinazioni* (risposta ancorata ai documenti)
- Possibilità di citare le fonti
- Adattabile a domini specifici

6.2 Retrieval Semantico con FAISS + SciBERT

FAISS (Facebook AI Similarity Search): libreria per nearest-neighbor efficiente su vettori densi.

- **IndexFlatIP**: inner product search su vettori normalizzati = cosine similarity
- Vettori: SciBERT (`allenai/scibert_scivocab_uncased`), 768 dimensioni
- SciBERT: BERT pre-trained su 1.14M paper scientifici (Semantic Scholar)

$$\text{Cosine}(q, d) = \frac{\vec{q} \cdot \vec{d}}{\|\vec{q}\| \cdot \|\vec{d}\|} \quad (\text{equivalente a inner product su vettori normalizzati})$$

Punti di forza: cattura relazioni semantiche e sinonimi

Limiti: impreciso su termini tecnici esatti (acronimi, nomi di algoritmi)

6.3 Retrieval Lessicale con BM25

BM25 (Best Match 25):

$$\text{BM25}(D, Q) = \sum_i \text{IDF}(q_i) \cdot \frac{f(q_i, D) \cdot (k_1 + 1)}{f(q_i, D) + k_1 \cdot \left(1 - b + b \cdot \frac{|D|}{\text{avgdl}}\right)}$$

$$\text{IDF}(q_i) = \log \frac{N - n(q_i) + 0.5}{n(q_i) + 0.5}$$

dove $k_1 = 1.5$, $b = 0.75$, N = dimensione corpus, $n(q_i)$ = doc frequency del termine.

- **Saturazione:** l'effetto della frequenza del termine raggiunge un plateau (parametro k_1)
- **Normalizzazione per lunghezza:** documenti lunghi non avvantaggiati ingiustamente (parametro b)
- Molto veloce (nessuna inferenza neurale)

Punti di forza: preciso su vocabolario tecnico (acronimi, termini specifici)

Limiti: nessuna comprensione semantica (sinonimi = miss), sensibile alla distribuzione del vocabolario

6.4 Hybrid Search: Reciprocal Rank Fusion (RRF)

Problema: FAISS e BM25 hanno punteggi non confrontabili (cosine $\in [0, 1]$ vs score BM25 $\in \mathbb{R}^+$).

Soluzione: combinare le *classifiche* (rankings) anziché i punteggi assoluti.

$$\text{RRF}(d) = \sum_{r \in R} \frac{1}{k + \text{rank}_r(d)}$$

dove R = insieme dei retriever, $k = 60$ (costante di smorzamento standard).

Perché funziona:

- Robusto all'eterogeneità degli score
- Favorisce documenti in cima a più classifiche (complementarietà)
- Nessun tuning manuale di pesi relativi
- Semplice e interpretabile

Procedura Lab 5:

1. BM25: top-20 per score lessicale
2. FAISS: top-20 per cosine similarity semantica
3. RRF: somma $\frac{1}{k + \text{rank}}$ per ogni documento nelle due liste
4. Re-ranking per score RRF finale, restituire top- k

```

1 from rank_bm25 import BM25Okapi
2 import faiss, numpy as np
3
4 # Inizializzazione
5 tokenized_docs = [doc.lower().split() for doc in documents]
6 bm25 = BM25Okapi(tokenized_docs)
7 faiss_index = faiss.IndexFlatIP(768)
8 faiss_index.add(embeddings_normalized)
9
10 def hybrid_retrieve(query, k=5, k_rrf=60):
11     # BM25 retrieval
12     bm25_scores = bm25.get_scores(query.lower().split())
13     bm25_top = np.argsort(bm25_scores)[: -1][:20]
14
15     # FAISS retrieval
16     q_emb = encode_and_normalize(query)
17     _, faiss_top = faiss_index.search(q_emb, 20)
18     faiss_top = faiss_top[0]
19
20     # Reciprocal Rank Fusion
21     rrf_scores = {}
22     for rank, idx in enumerate(bm25_top):
23         rrf_scores[idx] = rrf_scores.get(idx, 0) + 1/(k_rrf + rank +
24             1)
25     for rank, idx in enumerate(faiss_top):
26         rrf_scores[idx] = rrf_scores.get(idx, 0) + 1/(k_rrf + rank +
27             1)
28
29     sorted_docs = sorted(rrf_scores.items(), key=lambda x: x[1],
30         reverse=True)
31     return [documents[i] for i, _ in sorted_docs[:k]]

```

6.5 Componenti del Sistema (Lab 5)

Componente	Implementazione
Dataset	MaartenGr/arxiv_nlp (44.949 abstract NLP)
Encoder	allenai/scibert_scivocab_uncased (768-dim)
Semantic Index	FAISS IndexFlatIP
Lexical Retriever	BM25Okapi (rank-bm25)
Fusion	Reciprocal Rank Fusion ($k = 60$)
LLM Generator	microsoft/Phi-3.5-mini-instruct (3.8B param)
Max tokens gen.	300
Temperature	0.3 (quasi deterministico)

Prompt template:

```

1 system_prompt = (
2     "You are an expert NLP research assistant. "
3     "Answer ONLY based on the provided context. "
4     "Cite the papers you use."
5 )
6 context_text = "\n\n".join([doc[:200] for doc in retrieved_docs
7     [:5]])
8 context_text = context_text[:3000] # troncamento a 3000 char

```

```

8
9 messages = [
10     {"role": "system", "content": system_prompt},
11     {"role": "user", "content": f"Context:\n{context_text}\n\nQuestion: {query}"}
12 ]

```

7 Metriche di Valutazione (Lab 5)

7.1 Precision@K

$$P@K = \frac{|\text{documenti rilevanti nei top-}K|}{K}$$

Un documento è “rilevante” se il titolo contiene almeno una keyword del Gold Standard.
Gold Standard Lab 5 – 5 query NLP con keyword attese:

- “BERT / self-attention” → keyword: bert, attention, transformer, self-attention
- “word2vec” → keyword: word2vec, word embeddings, word vectors
- “machine translation” → keyword: translation, multilingual, parallel corpus
- “topic modeling” → keyword: topic, lda, latent dirichlet
- “named entity recognition” → keyword: ner, named entity, entity recognition

7.2 Context Relevance (CR)

$$CR = \frac{1}{|R|} \sum_{d \in R} \cos(\text{emb}(q), \text{emb}(d[:200]))$$

Allineamento semantico medio tra query e snippet dei documenti recuperati.

7.3 Answer Relevance (AR)

$$AR = \cos(\text{emb}(q), \text{emb}(\text{risposta}))$$

Misura se la risposta generata rimane centrata sulla query.

7.4 Faithfulness (FF)

$$FF = \frac{|\{w \in \text{answer} : w \in \text{context}\}|}{|\text{answer_content_words}|}$$

Proxy lessicale: quante parole di contenuto della risposta compaiono nel contesto recuperato. Alta FF = risposta ancorata ai documenti (meno allucinazioni).

7.5 Robustness

$$\text{Rob} = \frac{1}{|P|} \sum_{p \in P} \frac{|R_5(q) \cap R_5(p)|}{5}$$

Overlap tra risultati per query originale e parafrasi. Alta robustezza = retrieval stabile a riformulazioni.

8 Risultati Lab 5: Esperimento Multi-Corpus

8.1 Disegno Sperimentale

Variare la dimensione del corpus: $N \in \{1.000, 5.000, 15.000, 25.000, 44.949\}$

8.2 Tabella Risultati Completa

N	Metrica	Base (FAISS)	Hybrid (RRF)	Δ	% Migl.
1K	P@5	0.160	0.400	+0.240	+150%
	CR	0.716	0.718	+0.001	trascur.
	AR	0.697	0.684	-0.014	-2%
	FF	0.516	0.588	+0.072	+14%
5K	P@5	0.240	0.560	+0.320	+133%
	CR	0.724	0.714	-0.010	-1.4%
	AR	0.687	0.674	-0.014	-2%
	FF	0.572	0.667	+0.095	+16.6%
15K	P@5	0.160	0.520	+0.360	+225%
	CR	0.713	0.719	+0.007	+1%
	AR	0.686	0.667	-0.018	-2.6%
	FF	0.562	0.647	+0.084	+15%
25K	P@5	0.200	0.440	+0.240	+120%
	CR	0.720	0.730	+0.010	+1.4%
	AR	0.679	0.682	+0.003	+0.4%
	FF	0.560	0.587	+0.027	+4.8%
44.9K	P@5	0.120	0.360	+0.240	+200%
	CR	0.719	0.726	+0.007	+1%
	AR	0.678	0.666	-0.012	-1.8%
	FF	0.496	0.569	+0.074	+14.9%

8.3 Analisi Critica dei Risultati

1. P@5: miglioramento sistematico del Hybrid

L'approccio ibrido migliora consistentemente la P@5 su tutte le dimensioni corpus (+120% a +225%). BM25 recupera paper con acronimi tecnici esatti (NER, LDA, word2vec) che i soli embedding tendono a generalizzare.

2. P@5 non monotona con la dimensione del corpus

- Picco a $N = 5.000$ (P@5 hybrid = 0.560)
- Declino a $N = 44.949$ (P@5 hybrid = 0.360)

- Spiegazione: corpus piccolo = paper rilevanti concentrati, alta densità. Corpus grande = spazio raro, segnale di rilevanza diluito, più candidati competono

3. CR e AR: differenze minime

CR e AR sono costruiti su embedding SciBERT (usato da entrambi i sistemi). BM25 porta documenti lessicalmente precisi ma semanticamente più dispersi → leggero calo AR nel hybrid.

4. Faithfulness: miglioramento consistente

I documenti BM25 contengono i termini esatti della query → il LLM li riusa nell'answer → maggiore overlap lessicale answer-context. Questo è particolarmente utile nei domini tecnici dove il vocabolario è vincolato.

5. Robustness: declino catastrofico con la scala

N docs	Robustness (overlap@5)
1.000	0.233 (23.3%)
5.000	0.100 (10.0%)
15.000	0.033 (3.3%)
25.000	0.000 (0.0%)
44.949	0.033 (3.3%)

In corpus piccoli pochi paper dominanti → le parafrasi recuperano gli stessi. In corpus grandi lo spazio è denso di candidati vicini → una minima riformulazione cambia completamente i top-5. Soluzioni: query expansion, ensemble retriever, re-ranking gerarchico.

6. Precision per query (hybrid)

Query	1K	5K	15K	25K	44.9K
BERT/self-attention	0.20	0.40	0.40	0.00	0.20
word2vec	0.40	0.60	0.60	0.60	0.40
machine translation	0.20	0.40	0.60	0.60	0.40
topic modeling	0.40	0.60	0.20	0.20	0.20
named entity recog.	0.80	0.80	0.80	0.80	0.60
Media	0.40	0.56	0.52	0.44	0.36

“Named entity recognition” è il top performer stabile: acronimo NER molto specifico, BM25 lo identifica bene. “BERT/self-attention” instabile (0.00 a 25K): entrambi i termini sono ubiqui nel corpus NLP.

9 Confronto Approcci: Tabella Sinottica

Aspetto	TF-IDF/Jaccard (Lab 1-2)	BERTopic (Lab 4)	RAG (Lab 5)	Hybrid
Task	Misura similarità definizioni; recupero onomasiologico	Scoperta automatica topic	Risposta a domande su corpus	
Rappresentazione	Vettori sparsi (BoW / TF-IDF)	Embedding densi + riduzione dim	Embedding densi (FAISS) + sparsi (BM25)	
Semantica	Limitata (no sinonimi)	Alta (SentenceTransformer)	Alta semantica + alta precisione lessicale	
Scalabilità	Alta, veloce	Media (UMAP/HDBSCAN)	Dipende da corpus (best: 5K-15K)	
Output	Score similarità / synset	Lista topic + keywords	Risposta in linguaggio naturale	
Limite principale	Manca comprensione semantica profonda	Config errata → failure mode	Robustness degrada con scala	

10 Teoria: Embedding e Retrieval

10.1 Dense vs. Sparse Representation

Rappresentazioni sparse (TF-IDF, BM25):

- Dimensione = vocabolario (tipicamente 10^4 – 10^6)
- Quasi tutte le componenti sono zero
- Interpretabili: ogni dimensione = un termine
- Veloci, nessuna inferenza neurale

Rappresentazioni dense (BERT, SentenceTransformer):

- Dimensione fissa ridotta (128–1024)
- Tutte le componenti non-zero
- Catturano significato distribuito, sinonimi, relazioni implicite
- Più lente (require inferenza del modello)

10.2 Sentence Transformers

SentenceTransformer: modello BERT fine-tuned per produrre rappresentazioni a livello di frase.

- Training Siamese/triplet: frasi simili vicine, dissimili lontane nello spazio vettoriale
- Pooling sull'ultimo layer per ottenere vettore singolo per la frase
- Modelli comuni: `all-MiniLM-L6-v2` (384-dim, leggero), `all-mpnet-base-v2` (768-dim, alta qualità)

10.3 SciBERT vs. BERT generale

SciBERT (`allenai/scibert_scivocab_uncased`):

- Stessa architettura BERT (12 layer, 768-dim base)
- Vocabolario costruito su testi scientifici (formule, termini tecnici, notazione matematica)
- Pre-training su 1.14M paper da Semantic Scholar
- Superiore su IR scientifico rispetto a BERT generale

10.4 Cos'è HDBSCAN: Intuizione

HDBSCAN costruisce una gerarchia di cluster basata sulla densità:

1. Calcola la core distance di ogni punto (distanza al k -esimo vicino)
2. Costruisce il minimum spanning tree con distanze di mutua raggiungibilità
3. Condensa la gerarchia: rimuove rami che “si dividono” troppo rapidamente
4. Estrae cluster stabili con Excess of Mass (`eom`)
5. Punti non assegnati a nessun cluster = outlier (-1)

Vantaggio fondamentale per BERTopic: non si specifica quanti topic esistono, emergono dai dati.

11 Riepilogo Formule

Formula	Espressione	Uso
Jaccard (SimLex)	$ A \cap B / A \cup B $	Overlap lessicale definizioni
Cosine Similarity	$\vec{a} \cdot \vec{b} / (\vec{a} \cdot \vec{b})$	SimSem, FAISS, CR, AR
TF-IDF	$\text{tf}(t, d) \times \log(N/\text{df}(t))$	Vettorizzazione Lab1-2
BM25 score	$\sum_i \text{IDF}(q_i) \cdot \frac{f(q_i, D)(k_1+1)}{f(q_i, D) + k_1(1-b+b D /\text{avg})}$	Retrieval lessicale
RRF score	$\sum_r 1/(k + \text{rank}_r(d))$	Fusione ranking ibrido
c-TF-IDF	$\frac{f(t, c)}{ c } \cdot \log \frac{N}{\text{df}(t)}$	Topic keywords (BERTopic)
Precision@K	$ \text{rilevanti} \cap \text{top-}K / K$	Qualità retrieval
Faithfulness	$ \{w \in \text{ans} : w \in \text{ctx}\} / \text{ans} $	Grounding LLM
Robustness	$\frac{1}{ P } \sum_p R_5(q) \cap R_5(p) / 5$	Stabilità a parafrasi
Context Rel.	$\frac{1}{ R } \sum_d \cos(q, d[:200])$	Pertinenza contesto
Answer Rel.	$\cos(\text{emb}(q), \text{emb}(\text{ans}))$	Centramento risposta

12 Domande Tipiche da Esame

1. **Differenza tra SimLex e SimSem:** Jaccard misura overlap lessicale puro su insiemi di parole; TF-IDF coseno misura prossimità semantica in spazio vettoriale. Per concetti astratti SimSem > SimLex perché cattura sinonimi.
2. **Cos'è la ricerca onomasiologica?** Data una definizione, recuperare il synset WordNet corretto. Inverso della consultazione dizionario. Implementata con TF-IDF coseno tra definizione e glosse candidate.
3. **Perché i concetti concreti sono più recuperabili?** Il referente fisico vincola il vocabolario: persone diverse usano parole simili per descriverlo. I concetti astratti hanno alta variabilità e dipendenza culturale.
4. **Descrivere la pipeline BERTopic:** Documenti $\rightarrow$ Embedding (SentenceTransformer) $\rightarrow$ Riduzione dimensionale (UMAP) $\rightarrow$ Clustering (HDBSCAN) $\rightarrow$ Estrazione keyword (c-TF-IDF).
5. **Perché la Config 3 di BERTopic fallisce?** `min_cluster_size=150` crea cluster troppo grandi e tematicamente eterogenei. Il c-TF-IDF non riesce a estrarre segnale discriminativo e le stopwords emergono nel top-5.
6. **Cos'è RRF e perché si usa?** Reciprocal Rank Fusion combina ranking da retriever diversi (FAISS + BM25) senza richiedere normalizzazione degli score: $\sum_r 1/(k + \text{rank}_r(d))$. Sfrutta la complementarità di retrieval semantico e lessicale.
7. **Trend P@5 vs. corpus size:** non monotono, picco a 5K. Corpus troppo piccolo = pochi documenti rilevanti. Corpus troppo grande = segnale di rilevanza diluito in spazio denso di candidati.

8. **Perché la robustezza crolla a scala?** In corpus grandi lo spazio embedding è molto denso: una minima riformulazione della query sposta i top-k verso documenti simili ma diversi. Fix: query expansion, ensemble retrieval, re-ranking.
9. **Differenza BM25 vs. TF-IDF:** BM25 aggiunge saturazione della frequenza dei termini (k_1) e normalizzazione per lunghezza del documento (b), migliorando il ranking rispetto al semplice TF-IDF.
10. **Cosa misura la Faithfulness?** La percentuale di parole di contenuto della risposta LLM che compaiono nel contesto recuperato. Proxy lessicale per l'assenza di allucinazioni.